

```
!wget -O food.csv.gz https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz

--2026-04-25 16:07:04-- https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz
Resolving static.openfoodfacts.org (static.openfoodfacts.org)... 213.36.253.214
Connecting to static.openfoodfacts.org (static.openfoodfacts.org)|213.36.253.214|:443... connected.
HTTP request sent, awaiting response... 302 Moved Temporarily
Location: https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.products.csv.gz [following]
--2026-04-25 16:07:05-- https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.products.csv.gz
Resolving openfoodfacts-ds.s3.eu-west-3.amazonaws.com (openfoodfacts-ds.s3.eu-west-3.amazonaws.com)... 52.95.156.76, 16.12.18.46
Connecting to openfoodfacts-ds.s3.eu-west-3.amazonaws.com (openfoodfacts-ds.s3.eu-west-3.amazonaws.com)|52.95.156.76|:443... con
HTTP request sent, awaiting response... 200 OK
Length: 1258146794 (1.2G) [application/gzip]
Saving to: 'food.csv.gz'

food.csv.gz      100%[=====>]  1.17G  24.6MB/s   in 56s

2026-04-25 16:08:01 (21.5 MB/s) - 'food.csv.gz' saved [1258146794/1258146794]
```

```
import pandas as pd

cols = [
    'product_name',
    'categories_tags',
    'sugars_100g',
    'proteins_100g',
    'fat_100g'
]

df = pd.read_csv(
    "food.csv.gz",
    compression='gzip',
    sep='\t',
    usecols=cols,
    nrows=10000,
    low_memory=False
)

print(df.shape)
df.head()
```

(10000, 5)

	product_name	categories_tags	fat_100g	sugars_100g	proteins_100g
0	Limonade artisanale a la rose	NaN	NaN	NaN	NaN
1	M&M white	NaN	NaN	NaN	NaN
2	Chocolate n3	NaN	NaN	NaN	NaN
3	Pâte de fruits	NaN	NaN	NaN	NaN
4	Paleta gran reserva - Sierra nevada-	en:beverages-and-beverages-preparations,en:bev...	NaN	NaN	NaN

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df = df.dropna(subset=['sugars_100g', 'proteins_100g'])

df['sugars_100g'] = pd.to_numeric(df['sugars_100g'], errors='coerce')
df['proteins_100g'] = pd.to_numeric(df['proteins_100g'], errors='coerce')
df['fat_100g'] = pd.to_numeric(df['fat_100g'], errors='coerce')

df = df.dropna(subset=['sugars_100g', 'proteins_100g'])

df = df[
    (df['sugars_100g'] >= 0) & (df['sugars_100g'] <= 100) &
    (df['proteins_100g'] >= 0) & (df['proteins_100g'] <= 100)
]

print(df.shape)
df.head()
```

(3050, 5)

	product_name	categories_tags	fat_100g	sugars_100g	proteins_100g	
693	Pinto Bean	en:asian-style-ready-meal	10.20000	4.900000	17.500000	
694	Croquetas de bacalao	NaN	12.10000	1.900000	5.900000	
695	Keto & GF Granola	NaN	54.83871	3.225806	19.677419	
800	Ben's Pure Maple Cream	NaN	30.28000	37.720000	5.680000	
832	Guimauve chocolat smarties	NaN	19.00000	65.000000	5.700000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
import matplotlib.pyplot as plt

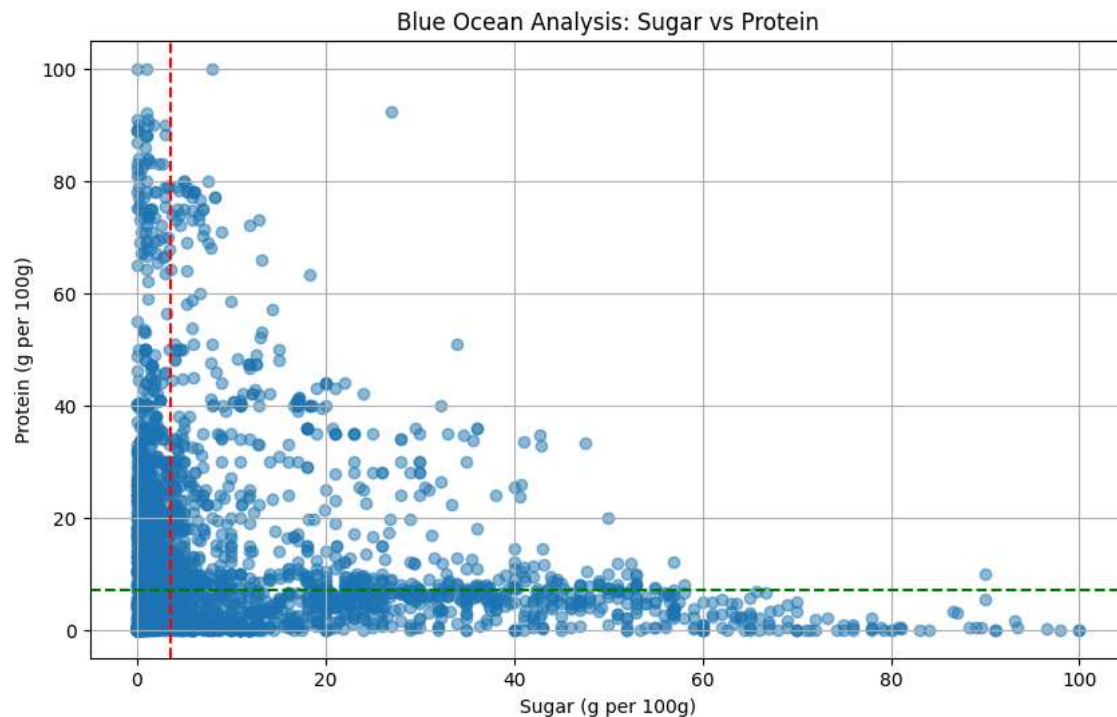
plt.figure(figsize=(10,6))

plt.scatter(
    df['sugars_100g'],
    df['proteins_100g'],
    alpha=0.5
)

plt.xlabel("Sugar (g per 100g)")
plt.ylabel("Protein (g per 100g)")
plt.title("Blue Ocean Analysis: Sugar vs Protein")

plt.axvline(x=df['sugars_100g'].median(), color='red', linestyle='--')
plt.axhline(y=df['proteins_100g'].median(), color='green', linestyle='--')

plt.grid(True)
plt.show()
```



```
protein_threshold = df['proteins_100g'].quantile(0.75)
sugar_threshold = df['sugars_100g'].quantile(0.25)

blue_ocean = df[
    (df['proteins_100g'] >= protein_threshold) &
    (df['sugars_100g'] <= sugar_threshold)
]

print("Protein threshold:", protein_threshold)
print("Sugar threshold:", sugar_threshold)
print("Blue Ocean size:", blue_ocean.shape)
```

blue_ocean.head()

Protein threshold: 18.56
Sugar threshold: 1.0
Blue Ocean size: (252, 5)

	product_name	categories_tags	fat_100g	sugars_100g	proteins_100g	
951	L'originel 1871	NaN	31.6	0.9	26.400000	
957	Poutargue de mullet	en:seafood,en:fishes-and-their-products,en:fis...	22.7	0.2	40.500000	
962	Saucisson sec au piment d'espelette	en:meats-and-their-products,en:meats,en:prepar...	33.3	0.7	24.100000	
1006	Pro-tf	NaN	0.0	0.0	22.448000	
1525	Csirke mellfilé	NaN	1.1	0.1	22.799999	

Next steps: [Generate code with blue_ocean](#) [New interactive sheet](#)

```
blue_ocean = blue_ocean.copy()
blue_ocean.loc[:, 'category'] = blue_ocean['categories_tags'].fillna("Unknown")
```

```
blue_ocean = blue_ocean.copy()

blue_ocean.loc[:, 'category'] = blue_ocean['categories_tags'].fillna("Unknown")

print(blue_ocean['category'].value_counts().head(10))
```

```
category
Unknown
en:dietary-supplements,en:bodybuilding-supplements,en:protein-powders
en:dietary-supplements
en:dietary-supplements,en:bodybuilding-supplements,en:protein-powders,en:protein-shakes
en:meats-and-their-products,en:meats,en:prepared-meats,en:cured-sausages,en:dry-sausages
en:snacks,en:sweet-snacks,en:cocoa-and-its-products,en:cocoa-and-chocolate-powders,en:cocoa-powders
en:seafood,en:crustaceans,en:shrimps,en:prawns,en:deep-water-pink-shrimps,en:cooked-prawns
en:snacks,en:salty-snacks,en:appetizers,en:chips-and-fries,en:meals,en:crisps
en:meats-and-their-products,en:beef-and-its-products,en:meals,en:meat-preparations,en:meats,en:skewers,en:beef,en:fish-and-meat-
en:plant-based-foods-and-beverages,en:plant-based-foods,en:breakfasts,en:cereals-and-potatoes,en:spreads,en:cereals-and-their-pr
Name: count, dtype: int64
```

```
def clean_category(tags):
    if pd.isna(tags):
        return "Other"

    tags = tags.lower()

    if "snack" in tags or "chips" in tags or "biscuits" in tags:
        return "Snacks"
    elif "beverages" in tags or "drink" in tags:
        return "Beverages"
    elif "dairy" in tags or "milk" in tags or "cheese" in tags:
        return "Dairy"
    elif "meat" in tags or "beef" in tags or "pork" in tags:
        return "Meat"
    elif "fish" in tags or "seafood" in tags or "crustacean" in tags:
        return "Seafood"
    elif "cereal" in tags or "breakfast" in tags:
        return "Cereals"
    elif "chocolate" in tags or "sweets" in tags:
        return "Sweets"
    elif "supplement" in tags or "protein" in tags:
        return "Protein Products"
    else:
        return "Other"

blue_ocean = blue_ocean.copy()
blue_ocean.loc[:, 'category'] = blue_ocean['categories_tags'].apply(clean_category)

blue_ocean['category'].value_counts()
```

category	count
Other	192
Meat	17
Protein Products	17
Snacks	9
Beverages	8
Seafood	5
Dairy	2
Sweets	1
Cereals	1

dtype: int64

```
top_category = blue_ocean['category'].value_counts().idxmax()

print(
    "Based on the data, the biggest market opportunity is in",
    top_category,
    "products targeting high-protein (>18g) and low-sugar (<1g) nutritional profiles."
)
```

Based on the data, the biggest market opportunity is in Other products targeting high-protein (>18g) and low-sugar (<1g) nutriti

```
filtered = blue_ocean[blue_ocean['category'] != "Other"]

top_category = filtered['category'].value_counts().idxmax()

protein_min = blue_ocean['proteins_100g'].min()
sugar_max = blue_ocean['sugars_100g'].max()

print(
    "Based on the data, the biggest market opportunity is in",
    top_category,
    "products targeting high-protein (>18g) and low-sugar (<1g) nutritional profiles."
)

print("\nSupporting metrics:")
print("Protein range:", protein_min, "to", blue_ocean['proteins_100g'].max())
print("Sugar range:", sugar_max)
```

Based on the data, the biggest market opportunity is in Meat products targeting high-protein (>18g) and low-sugar (<1g) nutritio

Supporting metrics:
Protein range: 18.56 to 100.0
Sugar range: 1.0

```
# Export full cleaned dataset
df.to_csv("clean_food_data.csv", index=False)

# Export Blue Ocean segment
blue_ocean.to_csv("blue_ocean_data.csv", index=False)

print("Export completed successfully:")
print("- clean_food_data.csv")
print("- blue_ocean_data.csv")
```

Export completed successfully:
- clean_food_data.csv
- blue_ocean_data.csv

```
from google.colab import files

files.download("clean_food_data.csv")
files.download("blue_ocean_data.csv")
```

